

Explicando o código

14 estágios · 6 pontos de bloqueio · 2 chamadas de modelo · ~1 700 linhas em `src/`

01 O caminho de um número

O código calcula todos os números do relatório e os entrega ao modelo como uma lista fechada. O modelo escreve o texto usando essa lista — ele não calcula, não arredonda e não estima. Depois, uma conferência automática lê a carta pronta e verifica número por número se cada um saiu dali.

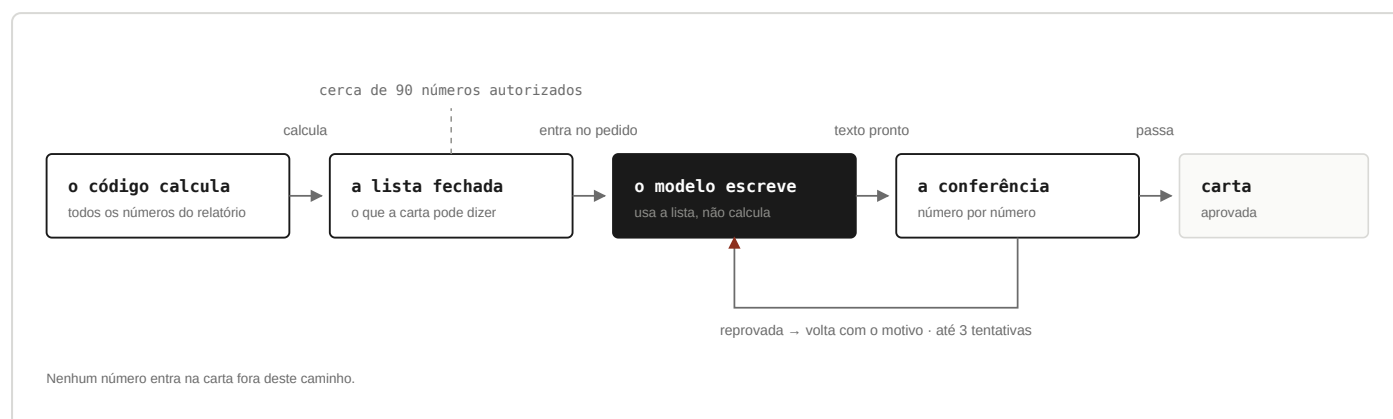
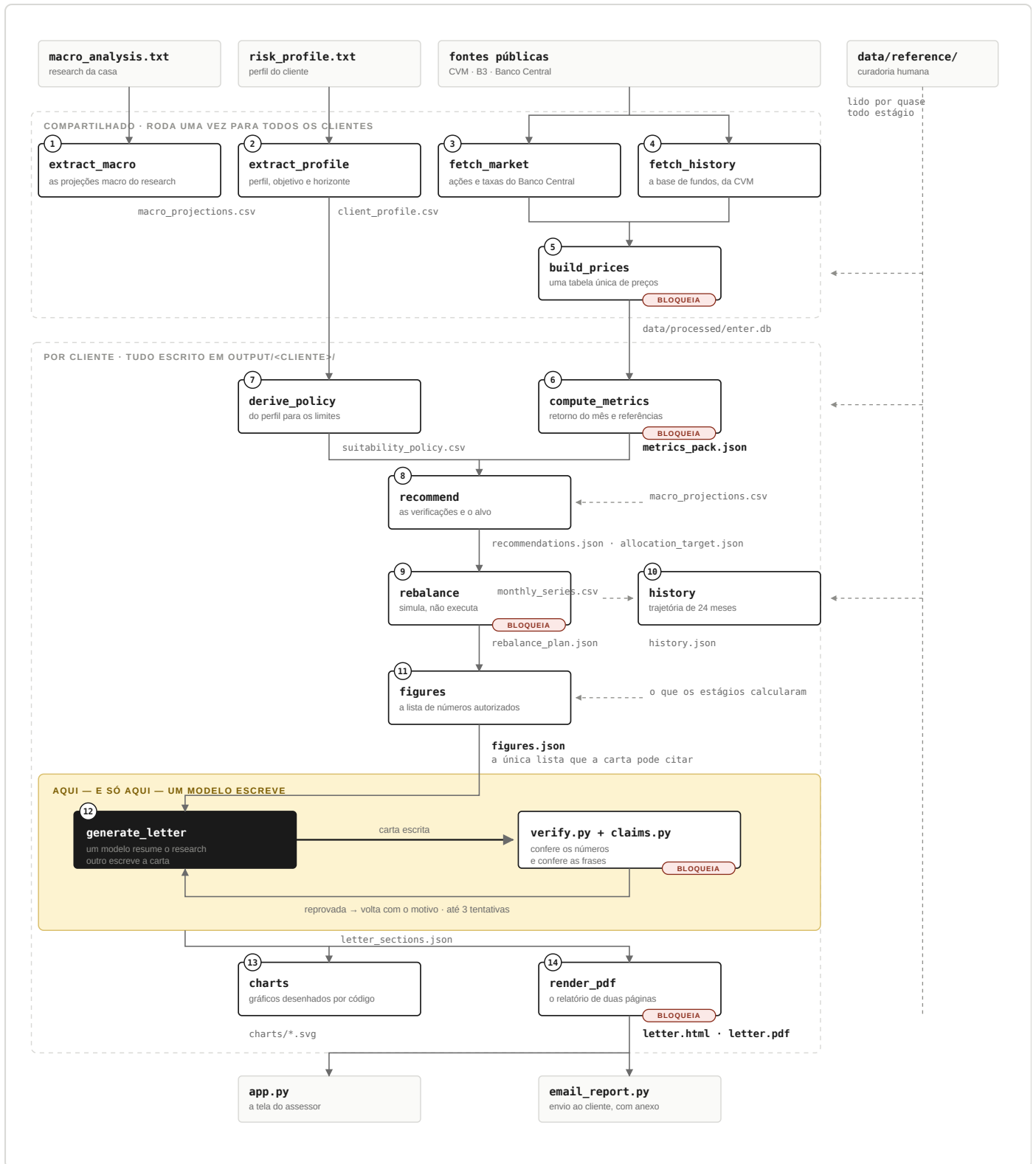


Figura 1. A lista é montada com os números já escritos exatamente como devem aparecer. É isso que torna a conferência simples e difícil de burlar: ou o número está na lista, ou a carta não sai.

02 O fluxo completo

Os catorze estágios, na ordem em que os dados correm. As faixas tracejadas separam o que é calculado uma vez para todo o universo do que é calculado por cliente — precificar o mercado inteiro a cada cliente seria desperdício.



□ estágio determinístico ■ chamada de modelo □ ponto de bloqueio
 → leitura de outro estágio, desenhada como toco

Figura 2. As ligações contínuas são o caminho principal; as tracejadas existem de verdade, mas desenhadas por inteiro deixariam o quadro ilegível — o nome do arquivo no toco diz o que entra ali. A numeração é a ordem em que os estágios rodam.

03 Os catorze estágios

Cada bloco abaixo corresponde a um nó da figura acima: o que entra, o que sai, o que o estágio faz e — onde existe — a decisão

que vale defender. Onde o exercício simplifica, a simplificação está declarada.

§1 **extract_macro.py** *(tira do relatório de research as projeções macro)*

COMPARTILHADO

SEM MODELO

```
lê      o relatório de research da casa
escreve macro_projections.csv
```

O relatório de research é uma peça de texto: cenário, argumentação, riscos. Mas o que uma decisão objetiva consegue consumir não é o argumento — é o **reflexo dele em projeções de variáveis macro**, como IPCA e Selic. Essas, sim, são números, e número vira parâmetro.

Por isso o motor de recomendação lê apenas a tabela de projeções, e não a prosa em volta. A prosa não é descartada: ela continua sendo o insumo do parágrafo de cenário da carta (§12). O que não acontece é uma decisão de alocação sair de uma frase de opinião.

A DECISÃO

A tabela aparece duas vezes no documento, e as duas cópias têm de bater. Os números que o texto corrido também cita são conferidos contra a tabela. **Onde os dois discordam, a tabela vence** e a divergência fica registrada em vez de ser resolvida em silêncio.

NUMA IMPLANTAÇÃO REAL

Estes números viriam de uma base oficial da XP, alimentada pelo trabalho dos economistas da casa a partir de várias fontes — uma fonte de verdade completa, com mais variáveis e um horizonte de projeção mais longo do que o relatório de um mês oferece.

extract_profile.py (transforma o perfil do cliente em parâmetros)

COMPARTILHADO

SEM MODELO

```
lê          o documento de perfil de risco do cliente
escreve client_profile.csv
```

Mesmo trabalho do estágio anterior, sobre outro documento: traduzir o que está exposto em texto em **variáveis parametrizáveis**, que o motor de recomendação consiga consumir. O documento de perfil é prosa e não contém nenhum número — nenhum percentual, nenhuma faixa, nenhum alvo.

Por simplicidade, o perfil assume três valores — Conservador, Moderado e Arrojado — e o horizonte assume outros três — Curto, Médio e Longo prazo —, cada um com uma correspondência fixa em anos. Cada valor lido fica gravado com a frase original que o produziu, para que a origem seja uma citação e não uma interpretação.

A DECISÃO

O documento do Albert diz *horizonte de médio a longo prazo*, que abrange duas das três opções. O sistema adota a mais curta — horizonte menor gera restrição mais apertada, então o erro fica do lado seguro — e registra a ambiguidade em vez de escondê-la.

A SIMPLIFICAÇÃO, ASSUMIDA

Seis caixas são muito menos do que se sabe sobre um cliente real, e muito menos granulares. O ponto aqui não é a taxonomia: é que os critérios sejam **parametrizados e objetivos**, em vez de ficarem no julgamento de quem estiver atendendo. O detalhamento de uma versão completa está na parte do relatório sobre o que eu faria com um mês de trabalho.

fetch_market.py (busca preços das ações e taxas do Banco Central)

COMPARTILHADO

BUSCA EXTERNA

```
busca      preços das ações e as séries de CDI, Selic e IPCA
```

Os preços e as séries do Banco Central foram puxados de fontes oficiais **por código**, não digitados à mão — o que torna o resultado reproduzível e auditável.

Como o exercício olha para um mês fechado no passado, o sistema não fica puxando esses dados ao vivo nem com determinada periodicidade: a busca foi feita uma vez, no início, e o resultado ficou salvo. Num sistema rodando mês a mês, essa busca seria recorrente.

A DECISÃO

Duas das ações da carteira mudaram de nome por incorporação. O provedor guarda o histórico sob o nome novo, então pedir pelo nome antigo — que era o vigente nas datas do relatório — não retorna nada. É um erro que não aparece como erro: a posição sairia do cálculo em silêncio.

NUMA IMPLANTAÇÃO REAL

Estes dados viriam da base oficial da XP, com informação em tempo real de todos os ativos, em vez de uma busca externa feita pelo próprio sistema.

fetch_history.py *(monta a base dos fundos, a partir da CVM)*

COMPARTILHADO

BASE IMUTÁVEL

escreve a base com as características e as cotações dos fundos

Fundo não tem cotação em provedor de mercado: quem publica é a CVM. Fui ao site da CVM, entendi como as bases de dados são estruturadas e identifiquei cada fundo da carteira pela sua descrição exata — CNPJ e código da CVM, e não pelo nome que aparece no extrato, que não é chave de nada.

Baixei as exportações com as características e as cotações dos fundos, e delas montei a base que abastece o sistema.

Nesta versão ela é imutável: os mesmos arquivos produzem sempre os mesmos números, e qualquer pessoa que rode o projeto chega exatamente ao mesmo resultado. Foi uma simplificação deliberada — o exercício só precisa de dados históricos.

A LIMITAÇÃO, ASSUMIDA

Isso não resolve o problema para frente. O sistema precisaria chamar as bases da CVM com determinada periodicidade para manter as cotações dos fundos atualizadas, em vez de depender de uma base congelada.

build_prices.py *(junta tudo numa tabela única de preços)*

COMPARTILHADO

BLOQUEIA

lê os dados de mercado, a base de fundos e o cadastro de instrumentos
escreve a tabela única de preços

Cada fonte entrega o preço de um jeito diferente; este estágio consolida tudo num formato só, de modo que o cálculo adiante não precise saber de onde cada número veio. É também onde moram os ajustes que separam um preço publicado do preço que de fato valia na data.

A DECISÃO

O preço publicado de um dos fundos vem arredondado a dois decimais. Sobre uma cota de R\$ 1,76, esse arredondamento distorce o retorno do mês **mais do que o próprio retorno** — então o preço é recalculado a partir do patrimônio do fundo, com a precisão cheia.

O BLOQUEIO

Os preços são conferidos contra um gabarito verificado à mão. Qualquer divergência encerra a execução aqui, antes que um preço errado chegue à carta de um cliente.

compute_metrics.py *(calcula quanto a carteira rendeu no mês)*

POR CLIENTE

BLOQUEIA

lê a tabela de preços e as posições do cliente
escreve metrics_pack.json — a fonte única de todo número da carta

Calcula o retorno do mês, quanto cada posição contribuiu para ele e a exposição real a renda variável. É a fonte única de verdade: **nenhum número da carta é recalculado depois daqui** — eles só são citados.

A DECISÃO

O desempenho é comparado com **duas referências, nunca misturadas**: o CDI responde se o risco pagou, e o IPCA responde se o dinheiro manteve o poder de compra. Fundir as duas numa referência só — meio juros, meio bolsa — produz um número que não descreve nem uma coisa nem outra. O Ibovespa aparece como contexto de mercado, não como referência.

O BLOQUEIO

Os resultados são conferidos contra um gabarito calculado à mão. Quando não existe gabarito para aquele cliente e mês, o sistema **diz em voz alta que a validação não rodou**, em vez de reportar uma aprovação que nunca aconteceu.

§7

derive_policy.py *(transforma o perfil nos limites que valem para o cliente)*

POR CLIENTE

SEM MODELO

```
lê          o perfil lido do documento e as tabelas de limites da casa
escreve     os limites em vigor para este cliente
```

É aqui que o perfil vira regra, e o estágio existe para manter dois tipos de afirmação separados. O documento diz *o que o cliente é*: moderado, buscando proteger o poder de compra, num horizonte médio. Todo limite numérico — teto de renda variável, teto por produto, caixa zero — é uma **calibração da casa**. Citar uma seção do documento ao lado desses números daria a eles uma precisão que a fonte não tem.

A DECISÃO

Cada regra sai declarando de onde veio. O relatório do estágio fecha com a conta: uma regra operacional, quatro de equivalência declarada, **zero lidas como número do documento** — porque o documento não tem números.

§8

recommend.py *(aponta o que está fora do lugar e qual deveria ser a alocação)*

POR CLIENTE

MOTOR DE DECISÃO

```
lê          os resultados do mês, os limites do cliente e as projeções macro
escreve     os pontos de atenção e o alvo de alocação
```

Cinco verificações: dinheiro parado em conta, papel vencido ainda em carteira, exposição a renda variável acima do teto, concentração excessiva num único produto e categoria fora do mandato. Cada achado sai com o fato medido, o limite que ele rompeu e de onde vem esse limite.

O alvo entre renda fixa e renda variável vem das projeções do próprio relatório de research, dentro do teto que o perfil permite. **Um modelo de linguagem nunca escolhe uma ação aqui** — ele pode, depois, redigir uma já decidida.

Como o alvo sai dos números. O perfil e o horizonte definem o *teto* de renda variável; o cenário macro decide *quanto desse teto* usar. Nenhuma das duas coisas é opinião: as duas saem de uma conta que roda igual toda vez. A conta abaixo é a que produziu o alvo do Albert, com os números que estavam nas entradas.

A CONTA, PASSO A PASSO — ALBERT, REFERÊNCIA ABRIL/2025

PROJEÇÕES DO RESEARCH (§1)

PIB 2025: 2,0% · 2026: 1,0%
IPCA 2025: 6,1% · 2026: 4,5%
Selic 2025: 15,50% · 2026: 12,50%

TETO DO PERFIL (§7)

Moderado × horizonte Médio
máximo 25% em renda variável

PREMISSAS DECLARADAS

dividend yield anual: 5%
sensibilidade: 10 p.p.
janela: mai/25 a dez/26

- 1 Quanto a renda variável deve render *acima da inflação*, ao ano
crescimento do PIB + dividend yield
2025: $2,0 + 5 = 7,00\%$ a.a. · 2026: $1,0 + 5 = 6,00\%$ a.a.
- 2 Quanto a renda fixa deve render *acima da inflação*, ao ano
 $(1 + \text{Selic}) \div (1 + \text{IPCA}) - 1$
2025: $1,155 \div 1,061 - 1 = 8,86\%$ a.a. · 2026: $1,125 \div 1,045 - 1 = 7,66\%$ a.a.
- 3 Acumula os dois pelos meses que faltam até o fim da janela
8 meses em 2025 + 12 meses em 2026
renda variável: 10,89% · renda fixa: 13,92%
- 4 Mede a diferença entre as duas
 $10,89 - 13,92$
-3,03 p.p. – a renda fixa está na frente nesta janela
- 5 Converte a diferença num fator entre 0 e 1
 $0,5 + \text{diferença} \div (2 \times \text{sensibilidade}) \rightarrow 0,5 + (-3,03 \div 20)$
0,348 – empate daria 0,5, ou metade do teto
- 6 Aplica o fator sobre o teto do perfil
 $25\% \times 0,348$
alvo renda variável 8,71% · renda fixa 91,29% · caixa 0%

A leitura é direta: o perfil do Albert permitiria até 25% em renda variável, mas o cenário projetado no relatório coloca a renda fixa três pontos à frente no acumulado até dezembro de 2026 — então o modelo usa pouco mais de um terço do teto permitido. **Se a diferença virasse a favor da bolsa, o alvo subiria pela mesma conta**, sem ninguém reabrir a discussão. E o teto nunca é ultrapassado: o macro decide o quanto, o perfil decide o limite.

A sensibilidade de 10 p.p. é o que define quão reativo o modelo é: uma diferença de 10 pontos a favor de uma classe leva o fator ao extremo. É uma premissa da casa, declarada como premissa — e é o tipo de parâmetro que, numa implantação real, sairia de um modelo de alocação calibrado, não de uma escolha.

O QUE ESTE MOTOR NÃO É

Construí um motor de decisão simples, e as limitações são conhecidas. Ele trata todo produto como renda fixa ou renda variável, o que apaga diferenças reais de risco dentro de cada balde. As regras de concentração são calibração, e **não derivam de um modelo comprovado de alocação de portfólio**. E ele não recomenda produtos específicos, porque não tem acesso ao cardápio completo de opções da casa.

O que o exercício quer mostrar não é este motor em particular. É que a recomendação saia de um **modelo determinístico** — idealmente com robustez e ciência por trás — em vez de nascer do viés individual de cada assessor ou de uma decisão arbitrária. Numa implantação real, este estágio seria um modelo de alocação sofisticado, mas ainda determinístico: capaz de enxergar os ativos que compõem cada fundo e a correlação entre eles, e conectado a um cadastro de produtos parametrizado.

§9

rebalance.py *(dimensiona as operações para chegar ao alvo)*

POR CLIENTE

BLOQUEIA

lê a carteira atual, o alvo de alocação e os pontos de atenção
escreve o plano de ajuste sugerido

Simula a transição da carteira de hoje para o alvo. **Nada é executado**. As vendas nomeiam ativos, porque as regras sabem exatamente quais posições rompem o limite; as compras nomeiam uma família de produto e um critério de crédito, não um produto.

A DECISÃO

A simulação confere que o patrimônio é preservado e que os alvos são atingidos. Mas isso cobre só os números: as categorias fora do mandato que permaneceram na carteira são reportadas à parte. **Uma alocação numericamente válida não é descrita como adequada em tudo.**

A LIMITAÇÃO, ASSUMIDA

A compra parar na família de produto é consequência direta do estágio anterior: sem o cardápio da casa parametrizado e conversando com um modelo de alocação, nomear um emissor seria um palpite com cara de recomendação. Com esse cardápio disponível, esta etapa nomearia o produto, o emissor e as condições — e a escolha continuaria saindo do modelo, não do assessor.

§10

history.py *(monta a trajetória de dois anos de cada investimento)*

POR CLIENTE

ENRIQUECE, NÃO BLOQUEIA

lê a base de cotações e as posições do cliente
escreve a trajetória mensal de cada posição

A carta fala do mês, mas o cliente também quer ver o caminho até aqui. Este estágio monta, para cada investimento da carteira, como ele se comportou nos últimos 24 meses.

Cada posição vira uma linha que **começa em 100 no primeiro mês da sua série**. É isso que torna comparáveis posições de tamanhos muito diferentes: uma linha que chega a 130 subiu 30%, seja uma ação de dez mil reais ou um fundo de cem mil. Sem essa base comum, a maior posição achataria todas as outras no mesmo gráfico.

A DECISÃO

Mês sem publicação **fica vazio** — nada é repetido do mês anterior nem estimado —, então uma série curta aparece curta, e o leitor vê onde o dado falta em vez de olhar para uma linha inventada. E as quantidades ficam constantes: a trajetória mostra como os preços se moveram, não aportes ou resgates que o cliente tenha feito no período. Isso é uma premissa, e ela aparece impressa no relatório em vez de ficar escondida no código.

Duas posições não têm como ter série completa, e o relatório declara as duas: um dos fundos só passa a ser publicado pela CVM a partir do seu cadastramento, e um CDB venceu antes da janela de dois anos.

§11 **figures.py** *(fecha a lista de números que a carta pode citar)*

POR CLIENTE

O MECANISMO CENTRAL

```
lê          tudo que os estágios anteriores calcularam
escreve figures.json – a lista de números autorizados
```

Todo número que a carta tem permissão de conter, já escrito exatamente como deve aparecer. São cerca de noventa. É este passo que torna a conferência posterior simples: ou o número que apareceu na carta está na lista, ou não está.

§12 **generate_letter.py** *(o modelo escreve o texto, e é conferido)*

AS DUAS CHAMADAS DE MODELO

ATÉ 3 TENTATIVAS

```
lê          a lista de números, os achados e o plano de ajuste
escreve o texto da carta, em seções
```

Dois níveis de modelo, de propósito: o mais barato condensa o relatório de research num parágrafo; o melhor escreve o texto que chega ao cliente e toca em cifras. A escolha de cada um é configurável, para que o custo possa ser revisto sem mexer no código.

A saída vem em sete campos fixos — abertura, desempenho, cenário, recomendações, cobertura, destaques e encerramento — e não em texto livre. Os títulos e as tabelas do documento vêm do template, de modo que **a estrutura da carta nunca depende de o modelo ter lembrado dela**.

A DECISÃO

A conferência não é só um porteiro: quando ela reprova, os motivos voltam nomeados para dentro do pedido seguinte e o modelo tenta de novo. São três tentativas; se a terceira não passar, nada é publicado.

§13 **charts.py** *(desenha os gráficos a partir dos números calculados)*

POR CLIENTE

SEM MODELO

```
escreve os gráficos, em arquivo de imagem
```

Os gráficos são desenhados por código, a partir do mesmo cálculo que alimenta a carta — nenhum deles é montado à mão, e nenhum contém um número que não tenha saído do §6.

Na versão final, porém, **a carta do cliente usa tabelas no lugar dos gráficos**. Em duas páginas, a tabela diz a mesma coisa em menos espaço e, ao contrário do gráfico, não perde informação quando o texto precisa encolher para caber no papel.

A DECISÃO

Nem no gráfico nem na tabela **a cor carrega informação sozinha**. Quem identifica é o rótulo, e a direção vem da seta e do sinal, não do verde e do vermelho. Assim o relatório continua legível impresso, em preto e branco e para um leitor com daltonismo. O amarelo da marca reprovou no teste de contraste e ficou restrito a filetes e à faixa do cabeçalho.

§14 **render_pdf.py** (monta o relatório final, em duas páginas)

POR CLIENTE

BLOQUEIA

lê o texto da carta e os números calculados
escreve letter.html · letter.pdf

O template é código, não saída de modelo: mesma entrada, mesmo documento. O modelo contribui apenas com a prosa; toda cifra e toda célula de tabela vêm do cálculo. Nada é buscado de fora na hora de abrir o arquivo — até as fontes vão embutidas, para que o documento não dependa do computador de quem o recebe.

O BLOQUEIO

O relatório tem duas páginas, e existe uma margem de ajuste declarada para o texto caber. Nenhuma tabela perde linhas em nenhum ponto dessa margem. **Conteúdo que não cabe é uma falha, não algo a cortar** — o estágio reprova e pede uma carta mais curta.

04 A tela do assessor

O pipeline roda por linha de comando, mas quem vai usar isso é um assessor com uma manhã de ligações pela frente. A interface existe para ele, e é **deliberadamente fina**: ela roda os mesmos estágios que o comando roda e mostra o que eles produziram. Nenhum cálculo acontece na tela — e é por isso que **a tela não consegue discordar da carta**.

O caminho é curto. O assessor entra, vê sua carteira de clientes, abre o cliente, lê o mês numa tabela, vê onde a carteira está e onde deveria estar, e gera a carta. O relatório sai em PDF e pode ser enviado por e-mail dali mesmo, com anexo.



Figura 3. Os dois botões são separados de propósito: conferir os números do mês não deveria custar uma chamada de modelo.

A fila mostra os outros clientes do cadastro em cinza, marcados como *não processados*. Só o Albert tem gabarito conferido à mão, então só ele é apurado — a tela declara isso em vez de exibir números que ninguém validou.

NUMA IMPLANTAÇÃO REAL

Esta tela seria o ponto de integração com os sistemas da XP — cadastro de clientes, posições em tempo real, e o envio pelos canais oficiais da casa, com o registro que a área de compliance exige. Aqui ela roda os estágios como processos locais e envia por um servidor de e-mail configurado.

05 Duas conferências, não uma

A primeira conferência verifica se todo número da carta existe e foi calculado. Ela não diz nada sobre as frases em volta — e uma carta pode citar apenas números corretos e ainda afirmar algo falso. Daí a segunda, que confronta o que as frases *afirmam* com os mesmos dados.

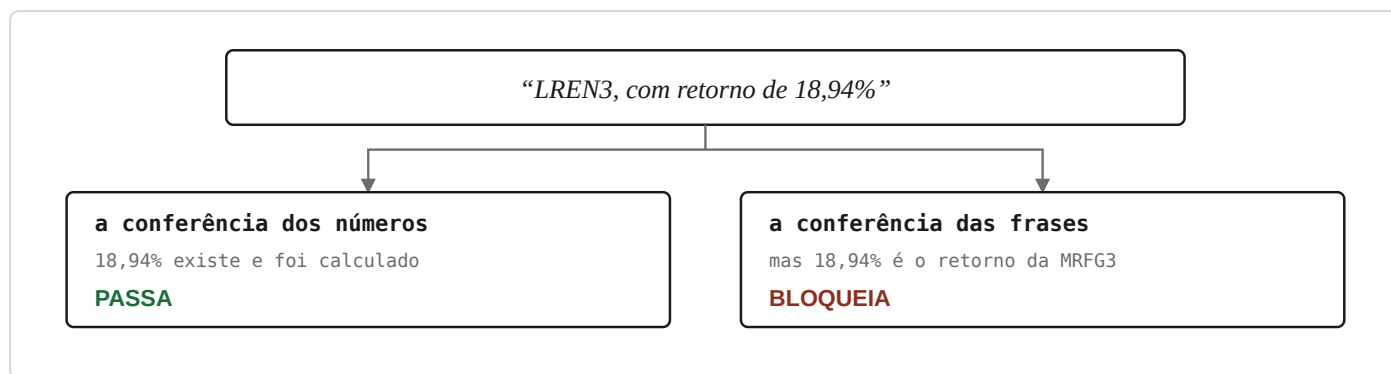


Figura 4. Todo número certo, frase errada — um número autorizado colado na posição errada passa em qualquer verificação que olhe só para dígitos.

Além da troca acima, a segunda conferência pega uma carta que diz que um ativo subiu quando ele caiu; que fala de uma operação no passado, como se já tivesse sido executada; que recomenda comprar ou vender algo que o plano não toca; que afirma ter superado o CDI quando não superou; e que declara a carteira adequada ao mandato enquanto há pontos de atenção em aberto.

Ela é deliberadamente conservadora. Uma frase ambígua é pulada e contada como não conferida, em vez de acusada — e o sistema reporta quantas frases conferiu e quantas deixou passar, em vez de sugerir que cobriu tudo.

06 Onde a execução para

Seis pontos em que o sistema se recusa a continuar. É a razão de cada estágio ser um programa separado: o primeiro que quebra interrompe tudo, em vez de deixar um número errado seguir até o cliente.

§5 · PREÇOS Gabarito dos preços Os preços consolidados são comparados com um gabarito verificado à mão.	§6 · CÁLCULO Gabarito da carteira Valores e retornos contra um gabarito calculado à mão. Sem gabarito, declara que a validação não rodou.
§9 · PLANO A conta do plano O patrimônio tem de ser preservado e os alvos atingidos exatamente na simulação.	§12 · NÚMEROS Conferência dos números Todo número da carta tem de constar da lista autorizada.
§12 · FRASES Conferência das frases O que as frases afirmam tem de bater com os dados. Reprovou, volta para o modelo.	§14 · DOCUMENTO 0 limite de duas páginas Se o texto não cabe na margem de ajuste declarada, reprovou — não corta conteúdo.

07 O que o modelo faz, e o que nunca faz

● Faz

- Condensa o relatório de research num parágrafo.
- Escreve o texto da carta em português, para um leitor sem formação em investimentos.
- Traduz conceito técnico em efeito prático: *retorno real* vira *ganho acima da inflação*; Selic, IPCA e CDI são explicados na primeira menção.
- Destaca uma frase por parágrafo.
- Reescreve quando a conferência reprova, com os motivos nomeados.

● Nunca faz

- Produz, calcula, arredonda ou estima um número.
- Escolhe uma ação. Comprar, vender e resgatar já estão decididos pelo motor de decisão.
- Nomeia um fundo, um papel ou um emissor para comprar.
- Decide a estrutura do documento — títulos e tabelas vêm do template.
- Toca nas posições, nos preços ou em qualquer dado de referência.

Projeto Enter · 14 estágios em `src/`, interface em `app.py`, mais uma suíte de testes que roda sobre uma cópia limpa do projeto.

Exercício de case independente. Cliente, assessor e carteiras são fictícios; as cartas geradas são demonstrações e não constituem recomendação de investimento.